ALGORITHMS LABORATORY

Theory and Exercises explained step by step

Author: SamuMGH04

WELCOME

Thanks for choosing these notes. This project is created with the aim to go more deep in C++ programming and explore advanced concepts.

LICENSE

These works are protected and distributed under the license
Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International

- **Attribution:** You must mention the original creator
- **Non Commercial:** You can't sell this material
- **No Derivatives:** You can't modify or change the text.

PROGRAMMAZIONE ORIENTATA AD OGGETTI (OOP), EREDITARIETA' , FUNZIONI VIRTUALI

Indice

1 Introduzione alla Programmazione Orientata agli Oggetti **4**

1.1 L'Analogia del Mondo Reale: Libri e Riviste 4

1.1.1 Attributi e Azioni 4

1.2 Fondamenti di Classi e Oggetti 5

1.2.1 La Classe come Progetto 5

1.2.2 L'Oggetto come Istanza 5

2 Gerarchie e il Concetto di Ereditarietà **6**

2.1 Relazioni tra Classi Simili 6

2.1.1 Diagramma delle Classi (Implementazione TikZ) 6

2.1.2 Analisi della Struttura 6

2.1.3 Analisi del Diagramma Gerarchico 7

2.2 Ereditarietà (Inheritance) 7

2.2.1 Meccanica dell'Ereditarietà 7

2.2.2 Terminologia Tecnica 7

2.3 Proprietà Matematiche dell'Ereditarietà 8

2.4 Il Principio di Sostituzione di Liskov (LSP) 8

2.5 Polimorfismo di Sottotipo (Subtype Polymorphism) 8

3 Implementazione dell'Ereditarietà in C++ **9**

3.1 La Classe Base: `work` 9

3.1.1 Spiegazione Dettagliata del Codice 10

3.2 La Classe Derivata: `book` 10

3.2.1 Spiegazione Dettagliata del Codice 11

3.3 Specializzazione per Riviste: `periodical` 11

3.3.1 Spiegazione Dettagliata del Codice 12

4 Funzioni Virtuali e Polimorfismo Dinamico **13**
4.1 Il problema della selezione dei metodi 13
4.1.1 Spiegazione Dettagliata del Codice 14
4.2 Classi Astratte e Funzioni Virtuali Pure 14
4.2.1 Spiegazione Dettagliata del Codice 14
4.3 Interfacce e Implementazioni 15

5 Polimorfismo e Gestione della Memoria **15**
5.1 Il Distruttore Virtuale 15
5.1.1 Spiegazione Dettagliata del Codice 16
5.2 Specializzazione della stampa: Il caso di `movie` 16
5.2.1 Spiegazione Dettagliata del Codice 17
5.3 Esempio di Utilizzo Polimorfico 17
5.3.1 Spiegazione Dettagliata del Codice 18

6 Collezioni Eterogenee e Smart Pointers **19**
6.1 Gestione Polimorfica con `std::unique_ptr` 19
6.1.1 Spiegazione Dettagliata del Codice 19
6.2 Iterazione e Invocazione Polimorfica 20
6.2.1 Spiegazione Dettagliata del Codice 20
6.3 Vantaggi del Design Polimorfico 20

7 Polimorfismo e Semantica del Passaggio dei Dati **21**
7.1 Il Problema dello "Slicing" (Affettamento) 21
7.1.1 Spiegazione Dettagliata del Codice 21
7.2 La Soluzione: Riferimenti e Puntatori 22
7.2.1 Spiegazione Dettagliata del Codice 22
7.3 Interfacce e Classi Astratte (Approfondimento) 22
7.4 Esercitazione: Progettazione di `report` 23
7.4.1 Osservazione Critica sulla Manutenibilità 23

8 Best Practices e Progettazione Avanzata **24**
8.1 Regole d'oro per i Distruttori 24
8.1.1 Spiegazione Dettagliata del Codice 24
8.2 Esercitazione Pratica: Pretty Print di un file CSV 25
8.2.1 Spiegazione Dettagliata del Codice 25
8.3 Verifica del Design: Estensibilità 26
8.4 Curiosità: cosa è `std::ostream&` 26
8.4.1 Scomposizione Tecnica 26
8.4.2 Il Concetto di "Stream" (Flusso) 27

8.4.3 Perché il Riferimento (&) è Obbligatorio? 27
8.4.4 Esempio di Utilizzo: Overloading dell'operatore « 28
8.4.5 Spiegazione Dettagliata del Codice 28

1 Introduzione alla Programmazione Orientata agli Oggetti

La programmazione moderna si avvale di paradigmi che cercano di approssimare il più possibile il codice alla realtà fenomenica. Uno dei più influenti è la Programmazione Orientata agli Oggetti (Object-Oriented Programming, OOP). L'obiettivo di questa sezione è comprendere come i concetti di "oggetto", "classe" e "interazione" si traducano in strutture software robuste e manutenibili.

1.1 L'Analogia del Mondo Reale: Libri e Riviste

Per comprendere l'essenza dell'OOP, è utile partire da esempi concreti. Consideriamo due entità comuni: un libro e una rivista. Se dovessimo analizzarli con occhio critico, noteremmo subito che possiedono sia tratti distintivi che tratti comuni.

1.1.1 Attributi e Azioni

Ogni entità nel mondo reale può essere descritta attraverso due lenti principali:

1. **Attributi (Attributes / Fields):** Rappresentano le caratteristiche fisiche o i dati che definiscono l'entità. Per un libro, questi potrebbero essere il titolo, l'autore, il numero di pagine o il codice ISBN. Gli attributi rispondono alla domanda: *"Com'è fatto l'oggetto?"*.

2. **Azioni (Actions / Methods):** Rappresentano il modo in cui l'oggetto agisce o il modo in cui noi interagiamo con esso. Ad esempio, "aprire il libro", "leggere una pagina" o "sfogliare la rivista". Le azioni rispondono alla domanda: *"Cosa fa l'oggetto?"* o *"Cosa si può fare con esso?"*.

Osservazione didattica: La distinzione tra attributi e azioni è fondamentale perché separa i **dati** (lo stato) dalle **funzioni** (il comportamento). In OOP, queste due componenti non sono più separate come nella programmazione procedurale, ma vengono fuse in un'unica entità coerente.

1.2 Fondamenti di Classi e Oggetti

Una volta compreso che gli oggetti reali hanno proprietà e comportamenti, dobbiamo definire come rappresentarli formalmente nel linguaggio C++.

1.2.1 La Classe come Progetto

Una **Classe** può essere immaginata come lo schema costruttivo o il "progetto" (blueprint) di una categoria di oggetti. Essa definisce formalmente quali saranno gli attributi e quali saranno le azioni che tutti i futuri oggetti appartenenti a quella categoria possiederanno.

- **Definizione:** Una classe descrive gli attributi (dati) e le azioni (metodi) comuni a tutti gli esemplari di quella categoria.
- **Esempio:** La classe "Libro" stabilisce che ogni libro deve avere un titolo, ma non specifica *quale* titolo.

1.2.2 L'Oggetto come Istanza

L' **Oggetto** è l'entità concreta creata seguendo le istruzioni della classe. In termini tecnici, diciamo che un oggetto è un'**istanza** di una classe.

- **Stato dell'Oggetto:** I dati contenuti negli attributi in un dato momento definiscono lo **stato** dell'oggetto. Se cambiamo il segnalibro in un libro, ne stiamo alterando lo stato.
- **Comportamento dell'Oggetto:** I **metodi** definiscono il comportamento. Essi sono le "funzioni" interne all'oggetto che possono operare sugli attributi per leggerli o modificarli, alterando quindi lo stato dell'oggetto stesso.

Sintesi dei collegamenti: Se la **Classe** è il concetto astratto di "Libro", l'**Oggetto** è la copia fisica de *"I Promessi Sposi"* che tieni in mano. La classe definisce la struttura; l'oggetto contiene i dati reali.

2 Gerarchie e il Concetto di Ereditarietà

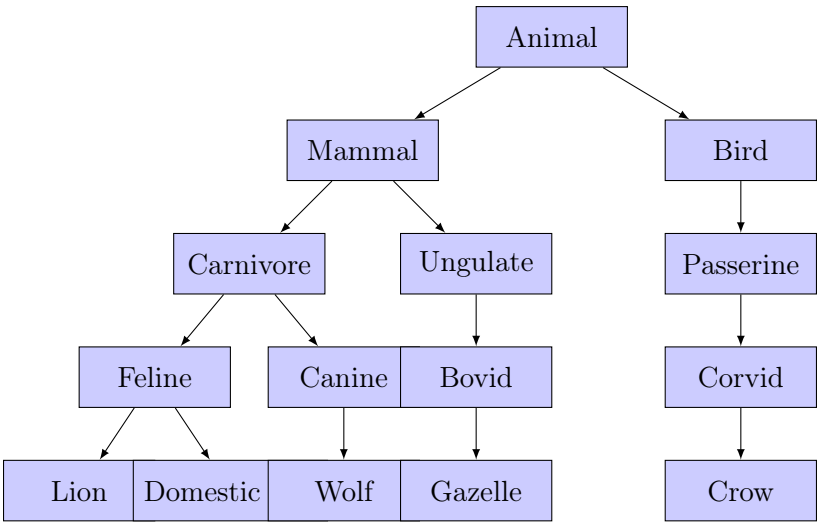
Dopo aver compreso la differenza tra classe (astrazione) e oggetto (istanza), è necessario analizzare come le classi possano essere messe in relazione tra loro per riflettere le gerarchie esistenti nel mondo reale.

2.1 Relazioni tra Classi Simili

Spesso ci troviamo a dover definire classi che condividono una base comune ma presentano specializzazioni differenti. Consideriamo l'esempio del regno animale proposto nel diagramma gerarchico:

2.1.1 Diagramma delle Classi (Implementazione TikZ)

L'albero seguente rappresenta la tassonomia degli oggetti. Si noti come la struttura sia un grafo diretto dove ogni nodo figlio eredita gli attributi e i metodi dal nodo genitore.



2.1.2 Analisi della Struttura

L'analisi del diagramma rivela proprietà fondamentali del design orientato agli oggetti applicato al caso di studio:

- **Ereditarietà Multi-livello**: Una classe come `Domestic Cat` (Gatto Domestico) non eredita solo da `Feline`, ma attraverso la catena gerarchica eredita anche da `Carnivore`, `Mammal` e `Animal`. Ciò significa che un'istanza di gatto avrà accesso a tutti i metodi definiti nella radice `Animal`.
- **Specializzazione**: Mentre la classe `Animal` definisce comportamenti generici (es. `eat()`, `sleep()`), le classi terminali (foglie) come `Lion` o `Wolf` implementano o specializzano comportamenti specifici.

- **Condivisione di Attributi**: Come evidenziato nel testo, un leone e un gatto domestico sono entità distinte, ma sono "simili" poiché condividono alcuni attributi e comportamenti derivanti dal loro antenato comune `Feline`.

Nota Tecnica: Nel diagramma sopra, abbiamo assunto una struttura ad albero semplice. Tuttavia le gerarchie di classi possono essere **DAG** (Grafi Diretti Aciclici) qualora sia permesso l'uso dell'ereditarietà multipla. Se l'ereditarietà multipla viene proibita o non utilizzata, la struttura degenera in un **albero**

2.1.3 Analisi del Diagramma Gerarchico

Radice (Root): In cima troviamo la classe generale `animal`.

Specializzazione progressiva: Scendendo lungo i rami, le classi diventano più specifiche. Da `animal` derivano `mammal` (mammifero) e `bird` (uccello).

Relazione "Is-A" (È un): Ogni nodo in basso eredita le proprietà del nodo superiore. Ad esempio, un `lion` (leone) "è un" `feline`, che a sua volta "è un" `carnivore`, che "è un" `mammal`, e infine "è un" `animal`.

Osservazione didattica: Un gatto domestico non è un leone, ma entrambi condividono attributi e comportamenti comuni perché appartengono alla stessa superfamiglia dei felini. L'obiettivo dell'ereditarietà è proprio quello di catturare queste somiglianze per evitare la duplicazione del codice.

2.2 Ereditarietà (Inheritance)

L'ereditarietà è il meccanismo che permette a una classe di acquisire le proprietà di un'altra.

2.2.1 Meccanica dell'Ereditarietà

In ambito OOP, se una classe **A** deriva (eredita) da una classe **B**, allora la classe **A** possiede automaticamente almeno tutti i comportamenti (metodi) e tutti gli attributi (campi) definiti in **B**.

- **Esempio concreto:** Un mammifero possiede tutti gli attributi di un animale; un gatto domestico possiede tutti gli attributi dei mammiferi.

2.2.2 Terminologia Tecnica

È fondamentale padroneggiare i sinonimi utilizzati in letteratura per descrivere questa relazione:

Classe Derivata / Sottoclasse (Subclass): La classe che eredita (Classe A). Viene definita anche come **specializzazione** della classe base. +1

Classe Base / Superclasse (Superclass): La classe da cui si eredita (Classe B).

2.3 Proprietà Matematiche dell'Ereditarietà

L'ereditarietà definisce quello che in matematica è chiamato un ordine parziale tra le classi. Questo legame è regolato da due proprietà fondamentali: +1

1. **Antisimmetria:** Se la classe **A** eredita da **B**, allora **B** non può ereditare da **A**. La gerarchia ha una direzione univoca: dalla generalizzazione alla specializzazione.

2. **Transitività:** Se **A** eredita da **B**, e una terza classe **C** eredita da **A**, allora **C** eredita automaticamente anche da **B**.

Di conseguenza, le gerarchie di classi sono strutturate come Grafi Diretti Aciclici (DAG). Tuttavia, se un linguaggio (o un design specifico) proibisce l'ereditarietà multipla, la struttura si semplifica diventando un albero. +1

2.4 Il Principio di Sostituzione di Liskov (LSP)

Uno dei pilastri dell'informatica è il *Liskov-Wing Substitution Principle.*

Definizione: Se la classe **A** eredita dalla classe **B**, allora qualsiasi frammento di codice che utilizza istanze di **B** deve poter funzionare correttamente anche se tali istanze vengono sostituite con oggetti della classe **A**.

Significato profondo: Questo principio garantisce che una sottoclasse sia sempre compatibile con l'interfaccia della sua classe base. Gli oggetti di una classe base possono essere sempre sostituiti da oggetti di classi derivate senza alterare la correttezza del programma.

2.5 Polimorfismo di Sottotipo (Subtype Polymorphism)

Il termine "polimorfismo" deriva dal greco e significa "molte forme".

Concetto: Un oggetto di una classe base può "assumere la forma" di una qualsiasi delle sue sottoclassi. Ad esempio, una variabile di tipo `Mammal` può contenere un cane, un gatto o una persona, ma non può contenere un sasso o un uccello (poiché non appartengono alla stessa gerarchia).

In C++: Una variabile dichiarata con il tipo della classe base **B** può riferirsi a un oggetto di classe **B** oppure a qualsiasi oggetto di una classe derivata da **B**.

3 Implementazione dell'Ereditarietà in C++

In questa sezione analizzeremo come tradurre i concetti teorici di gerarchia in codice C++ reale, utilizzando l'esempio di un catalogo di opere editoriali. Vedremo come definire una classe base e come estenderla per creare tipi più specifici.

3.1 La Classe Base: `work`

La classe `work` rappresenta l'astrazione di un'opera generica. Ogni elemento del nostro catalogo, che sia un libro o una rivista, possiede un identificativo e un titolo; pertanto, questi attributi risiedono nella classe base per essere condivisi.

Listing 1: Definizione della classe base work

```
class work {
public:
// Costruttore predefinito esplicitato come default
work() = default;

// Costruttore di copia esplicitato come default
work(work const& w) = default;

// Costruttore personalizzato con std::string_view per efficienza
work(std::string_view id, std::string_view title)
    : id_{id}, title_{title} {}

// Metodo getter per l'ID (costante, non modifica l'oggetto)
std::string const& id() const { return id_; }

// Metodo getter per il titolo (costante)
std::string const& title() const { return title_; }
private:
std::string id_;     // Identificativo univoco dell'opera
std::string title_; // Titolo dell'opera
};
```

3.1.1 Spiegazione Dettagliata del Codice

- **Gestione dei Costruttori**: L'utilizzo di `= default` istruisce il compilatore a generare le implementazioni standard per il costruttore vuoto e quello di copia, risparmiando scrittura di codice ridondante.
- **Efficienza con `std::string_view`**: Introdotto in C++17, `std::string_view` è un oggetto "leggero" che permette di passare stringhe ai costruttori evitando copie inutili della memoria, migliorando le prestazioni rispetto al classico `std::string` o `char`.
- **Incapsulamento**: I dati (`id_` e `title_`) sono dichiarati `private`. Questo significa che non sono accessibili direttamente dall'esterno, ma solo attraverso le funzioni pubbliche (metodi getter), garantendo l'integrità dell'oggetto.

3.2 La Classe Derivata: `book`

La classe `book` eredita da `work` e aggiunge attributi specifici per i libri, come l'autore e l'anno di pubblicazione. +2

Listing 2: Definizione della classe derivata book

```
// La classe book eredita pubblicamente da work
class book : public work {
public:
// Costruttore predefinito: inizializza la base e i propri membri
book() : work{}, author_{}, pubyear_{0} {}

// Costruttore di copia predefinito
book(book const& b) = default;

// Costruttore completo: delega alla classe base l'inizializzazione
di ID e Titolo
book(std::string_view id, std::string_view title,
     std::string_view author, int pubyear)
  : work{id, title}, author_{author}, pubyear_{pubyear} {}

// Getter specifico per l'autore
std::string const& author() const { return author_; }

// Getter specifico per l'anno di pubblicazione
int pubyear() const { return pubyear_; }
```

```
private:
std::string author_; // Nome dell'autore
int pubyear_;        // Anno di pubblicazione
};
```

3.2.1 Spiegazione Dettagliata del Codice

- **Sintassi di Ereditarietà**: La dicitura : `public work` indica che `book` è una sottoclasse di `work`. L'ereditarietà `public` garantisce che tutti i metodi pubblici di `work` (come `id()` e `title()`) siano accessibili anche agli utenti della classe `book`.
- **Inizializzazione della Classe Base**: È fondamentale notare come il costruttore di `book` richiami `workid, title` nella sua lista di inizializzazione. Un oggetto derivato contiene al suo interno un oggetto della classe base, che deve essere costruito correttamente prima dei membri della classe figlia.
- **Specializzazione**: `book` estende le funzionalità di `work` aggiungendo `author_` e `pubyear_`, dimostrando come l'ereditarietà permetta di riutilizzare il codice della base aggiungendo solo ciò che è specifico.

3.3 Specializzazione per Riviste: `periodical`

Analogamente a `book`, creiamo la classe `periodical` per gestire riviste o giornali, che richiedono informazioni su volumi e date di uscita. +3

Listing 3: Definizione della classe derivata periodical

```
class periodical : public work {
public:
// Inizializzazione predefinita dei campi numerici e testuali
periodical() : work{}, volume_{0}, number_{0}, date_{} {}

periodical(periodical const& p) = default;

// Costruttore che inizializza base e campi specifici
(volume, numero, data)
periodical(std::string_view id, std::string_view title,
           int volume, int number, std::string_view date)
  : work{id, title}, volume_{volume}, number_{number}, date_{date} {}

int volume() const { return volume_; }
int number() const { return number_; }
```

```
std::string const& date() const { return date_; }
private:
int volume_;        // Numero del volume
int number_;        // Numero dell'edizione
std::string date_; // Data di pubblicazione (es. "Gennaio 2024")
};
```

3.3.1 Spiegazione Dettagliata del Codice

- **Flessibilità della Gerarchia**: `periodical` dimostra come una singola classe base (`work`) possa avere più discendenti diversi tra loro. Sia `book` che `periodical` "sono dei" `work`, ma gestiscono dati differenti.

- **Consistenza dei Dati**: Anche qui, la responsabilità di gestire `id` e `title` è lasciata interamente alla classe `work`, evitando duplicazione di logica e variabili.

4 Funzioni Virtuali e Polimorfismo Dinamico

Finora abbiamo visto come ereditare attributi e metodi. Tuttavia, un problema sorge quando vogliamo che una classe derivata ridefinisca un comportamento della classe base in modo che il programma "scelga" la versione corretta a runtime (durante l'esecuzione).

4.1 Il problema della selezione dei metodi

Supponiamo di voler stampare i dettagli di un'opera. Se abbiamo un puntatore a `work` che punta in realtà a un oggetto `book`, il compilatore, di default, chiamerà il metodo di `work`. Per evitare questo comportamento e usare la versione specifica di `book`, introduciamo la parola chiave `virtual`.

Listing 4: Utilizzo di funzioni virtuali

```
class work {
public:
// La parola chiave virtual permette l'override nelle classi derivate
virtual void print() const {
std::cout << id_ << ":" << title_ << std::endl;
}
// ... resto della classe ...
};

class book : public work {
public:
// Ridefinizione (override) del metodo print
// La keyword 'override' assicura che stiamo davvero sovrascrivendo
un metodo virtuale
void print() const override {
std::cout << id() << ":" << title() << "," << author_ << std::endl;
}
// ... resto della classe ...
};
```

4.1.1 Spiegazione Dettagliata del Codice

- **La Keyword** `virtual`: Quando dichiariamo un metodo come `virtual` nella classe base, stiamo dicendo al C++: "Non decidere ora quale funzione chiamare; aspetta di vedere che tipo di oggetto è realmente presente a runtime".
- **L'identificatore** `override`: Introdotto nel C++11, non è obbligatorio ma è una *best practice* fondamentale. Se sbagliamo a scrivere il nome del metodo o i parametri nella classe derivata, il compilatore ci segnalerà un errore invece di creare silenziosamente una nuova funzione.
- **Late Binding (Collegamento Posticipato)**: Questo meccanismo è ciò che permette al polimorfismo di funzionare. Il programma consulta una tabella interna (chiamata *vtable*) per trovare l'indirizzo della funzione corretta.

4.2 Classi Astratte e Funzioni Virtuali Pure

A volte, una classe base è così generica che non ha senso istanziarla. Ad esempio, che cos'è un "Animale" generico? Ha senso creare un oggetto `work` che non sia né un libro, né un film, né una rivista? In questi casi usiamo le funzioni virtuali pure.

Listing 5: Esempio di Classe Astratta

```
class work {
public:
// Funzione virtuale pura: non ha implementazione nella classe base
virtual void print() const = 0;

// Un distruttore virtuale e essenziale nelle classi base
virtual ~work() = default;
};
```

4.2.1 Spiegazione Dettagliata del Codice

- **Sintassi** `= 0`: Questa sintassi trasforma la funzione virtuale in una funzione virtuale pura. La classe che la contiene diventa automaticamente una Classe Astratta.
- **Proprietà delle Classi Astratte**: Non è possibile creare oggetti di una classe astratta (es. `work w;` darebbe errore). È possibile però creare puntatori o riferimenti a essa per sfruttare il polimorfismo.
- **Obbligo per le sottoclassi**: Qualsiasi classe che erediti da `work` deve implementare obbligatoriamente il metodo `print()`, altrimenti diventerà anch'essa una classe astratta.

4.3 Interfacce e Implementazioni

Il concetto di classe astratta ci permette di separare l'interfaccia (cosa l'oggetto può fare) dall'implementazione (come lo fa).

Osservazione didattica: Questo approccio è alla base del design del software moderno. Ci permette di scrivere funzioni che lavorano con puntatori a `work`, e tali funzioni continueranno a funzionare correttamente anche se in futuro aggiungeremo nuovi tipi di opere (es. `movie` o `software`), senza dover cambiare una singola riga di codice della funzione principale

5 Polimorfismo e Gestione della Memoria

Il polimorfismo dinamico introduce delle responsabilità aggiuntive per il programmatore, specialmente per quanto riguarda la distruzione degli oggetti. Se gestiamo oggetti derivati tramite puntatori alla classe base, dobbiamo assicurarci che l'intera struttura dell'oggetto venga rimossa correttamente dalla memoria.

5.1 Il Distruttore Virtuale

Una regola aurea della programmazione in C++ stabilisce che ogni classe base destinata ad essere ereditata debba avere un distruttore dichiarato come `virtual`.

Listing 6: Esempio di distruttore virtuale nella classe base

```
class work {
public:
work() = default;

// Distruttore virtuale: fondamentale per il polimorfismo
virtual ~work() = default;

virtual void print() const = 0; // Funzione virtuale pura
private:
std::string id_;
std::string title_;
};
```

5.1.1 Spiegazione Dettagliata del Codice

- **Perché `virtual`?**: Se abbiamo un puntatore `work* ptr = new book();` e chiamiamo `delete ptr;`, se il distruttore non fosse virtuale, verrebbe invocato solo il distruttore di `work`. Gli attributi specifici di `book` (come la stringa `author_`) non verrebbero distrutti correttamente, causando una perdita di memoria (*memory leak*).

- **Sintassi `= default`**: Come per i costruttori, indichiamo al compilatore di generare il distruttore standard, ma la parola chiave `virtual` garantisce che il meccanismo della *vtable* (tabella dei metodi virtuali) venga utilizzato per trovare il distruttore della classe più derivata.

5.2 Specializzazione della stampa: Il caso di `movie`

Il materiale introduce una nuova classe, `movie`, per mostrare come il polimorfismo permetta di gestire diverse implementazioni dello stesso metodo virtuale `print()`.

Listing 7: Implementazione della classe movie

```
class movie : public work {
public:
movie() : work{}, director_{}, year_{0} {}

movie(std::string_view id, std::string_view title,
      std::string_view director, int year)
  : work{id, title}, director_{director}, year_{year} {}

// Implementazione specifica del metodo print per i film
void print() const override {
    std::cout << "Film: " << title() << " [" << year_ << "]"
              << " Diretto da: " << director_ << std::endl;
}
private:
std::string director_;
int year_;
};
```

5.2.1 Spiegazione Dettagliata del Codice

- **Override di `print()`**: La classe `movie` fornisce la sua versione del metodo `print()`. Nota l'uso di `title()` (il getter pubblico della classe base) poiché l'attributo `title_` è privato in `work` e non accessibile direttamente dalle sottoclassi.
- **Coerenza Gerarchica**: Anche se un film ha attributi diversi da un libro (regista vs autore), entrambi possono essere trattati uniformemente come `work`.

5.3 Esempio di Utilizzo Polimorfico

Vediamo ora come un singolo contenitore possa ospitare oggetti diversi e manipolarli in modo uniforme.

Listing 8: Esempio di contenitore polimorfico

```
int main() {
// Un vettore di puntatori alla classe base
std::vector<std::unique_ptr<work>> catalog;

// Aggiungiamo oggetti di tipo diverso
catalog.push_back(std::make_unique<book>("B01",
"I Malavoglia", "Verga", 1881));
catalog.push_back(std::make_unique<movie>("M99", "Inception",
"Nolan", 2010));

// Iterazione polimorfica
for (const auto& item : catalog) {
    item->print(); // Chiama la versione corretta a runtime!
}

return 0;
}
```

Output Atteso

```
B01: I Malavoglia, Verga
Film: Inception [2010] Diretto da: Nolan
```

5.3.1 Spiegazione Dettagliata del Codice

- `std::unique_ptr`: È un "puntatore intelligente" che gestisce automaticamente la memoria. Quando il vettore esce dallo scope, gli oggetti vengono distrutti automaticamente. È lo strumento standard in C++ moderno per gestire il polimorfismo.

- `const auto&:` L'uso di const auto& è la "best practice" per gestire oggetti polimorfici in modo efficiente e sicuro, specialmente all'interno di cicli o interfacce generiche.

  1. Perché usare il riferimento (&)? Evita l'Object Slicing: Se usi auto per valore, il compilatore crea una copia dell'oggetto base, "tagliando via" la parte derivata e perdendo il comportamento polimorfico. Il riferimento (&) mantiene il legame con l'istanza originale e la sua vtable (meccanismo usato da compilatore C++ per supportare il polimorfismo dinamico)
     Preserva il Dinamismo: Permette al programma di invocare i metodi virtual corretti della classe derivata anche se il tipo dedotto è la classe base.

  2. Perché usare const? Efficienza: Evita copie costose di oggetti pesanti. Sicurezza: Garantisce che l'oggetto non venga modificato durante l'ispezione. Lifetime Extension: Permette di legare il riferimento anche a oggetti temporanei (rvalue, ovvero sta per "right value" e identifica i valori che solitamente si trovano a destra di un operatore di assegnamento), estendendone la vita fino alla fine dello scope del riferimento.

- **Il Ciclo** `for`: Nonostante `item` sia visto dal compilatore come un puntatore a `work`, grazie alla parola chiave `virtual`, durante l'esecuzione verrà invocato il metodo `print()` di `book` per il primo elemento e quello di `movie` per il secondo.

6 Collezioni Eterogenee e Smart Pointers

Uno dei vantaggi principali dell'ereditarietà e del polimorfismo è la possibilità di gestire oggetti di tipi diversi (ma con una base comune) all'interno di un'unica struttura dati. In C++ moderno, questo si realizza attraverso l'uso di `std::unique_ptr` e del contenitore `std::vector`.

6.1 Gestione Polimorfica con `std::unique_ptr`

Il problema fondamentale nel gestire una lista di "opere" (che possono essere libri, film o riviste) è che ogni oggetto ha dimensioni diverse in memoria. Non possiamo creare un `std::vector<work>`, perché ciò causerebbe il fenomeno dello *slicing* (taglio), dove le parti specifiche di un libro o di un film verrebbero rimosse per farle entrare nello spazio di un'opera generica. La soluzione è memorizzare dei puntatori.

Listing 9: Creazione di un catalogo eterogeneo

```
#include <vector>
#include <memory> // Necessario per std::unique_ptr

// Creazione di un vettore di puntatori intelligenti alla classe base
std::vector<std::unique_ptr<work>> catalog;

// Inserimento di oggetti di classi derivate diverse
catalog.push_back(std::make_unique<book>
("001", "Il Nome della Rosa", "Umberto Eco", 1980));
catalog.push_back(std::make_unique<movie>
("002", "Interstellar", "Christopher Nolan", 2014));
catalog.push_back(std::make_unique<periodical>
("003", "Le Scienze", 500, 12, "Dicembre 2023"));
```

6.1.1 Spiegazione Dettagliata del Codice

- `std::unique_ptr<work>`: È un puntatore che "possiede" l'oggetto a cui punta. Quando il puntatore viene rimosso dal vettore o il vettore viene distrutto, `unique_ptr` chiama automaticamente il distruttore dell'oggetto, evitando perdite di memoria.

- `std::make_unique`: Questa funzione è il modo più sicuro ed efficiente per creare un oggetto dinamicamente. Alloca la memoria e costruisce l'oggetto in un unico passaggio.

- **Astrazione**: Il vettore vede solo dei `work`. Non ha bisogno di sapere che sta contenendo un libro o un film; si fida dell'interfaccia definita nella classe base.

6.2 Iterazione e Invocazione Polimorfica

Una volta riempito il catalogo, possiamo elaborare tutti gli elementi in modo uniforme.

Listing 10: Ciclo di stampa polimorfico

```
// Uso di un reference costante per iterare senza copiare i puntatori
for (const auto& item : catalog) {
// Chiamata al metodo virtuale
item->print();
}
```

Output Atteso (Simulazione)

```
001: Il Nome della Rosa, Umberto Eco
Film: Interstellar [2014] Diretto da: Christopher Nolan
003: Le Scienze, Vol. 500, N. 12 (Dicembre 2023)
```

6.2.1 Spiegazione Dettagliata del Codice

- **Riferimento costante (`const auto&`)**: Poiché `unique_ptr` non può essere copiato (il possesso è unico), dobbiamo iterare usando un riferimento. `const` garantisce che non modificheremo i puntatori nel vettore durante il ciclo.

- **Risoluzione Dinamica**: Nonostante ogni elemento nel ciclo sia trattato come un `work*`, il sistema di runtime del C++ consulta la *vtable* di ogni singolo oggetto e salta all'implementazione corretta di `print()` (quella di `book`, `movie` o `periodical`).

6.3 Vantaggi del Design Polimorfico

Questo approccio rispetta uno dei principi cardine dell'ingegneria del software: il Principio Open/Closed (Aperto alle estensioni, ma chiuso alle modifiche).

Osservazione didattica: Se un domani volessimo aggiungere un nuovo tipo di opera, ad esempio `software`, dovremmo solo creare la classe `software` ereditando da `work` e implementando `print()`. Il codice del ciclo di stampa visto sopra non dovrà essere cambiato di una virgola. Questa è la vera potenza del polimorfismo dinamico: la capacità di scrivere codice che rimane valido anche a fronte di evoluzioni future del sistema.

7 Polimorfismo e Semantica del Passaggio dei Dati

Uno dei concetti più ostici per chi approccia il C++ è capire come il polimorfismo interagisca con il modo in cui passiamo gli oggetti alle funzioni. La scelta tra passare un oggetto "per valore", "per riferimento" o "per puntatore" non è solo una questione di efficienza, ma di correttezza logica.

7.1 Il Problema dello "Slicing" (Affettamento)

Quando passiamo un oggetto di una classe derivata a una funzione che accetta un parametro della classe base per valore, si verifica un fenomeno degradante chiamato *slicing*.

Listing 11: Esempio di Slicing (ERRORE COMUNE)

```
// Funzione che accetta un oggetto PER VALORE
void print_info(work w) {
w.print();
}

int main() {
book b{"001", "Dune", "Frank Herbert", 1965};
print_info(b); // Errore logico: b viene "affettato"
}
```

7.1.1 Spiegazione Dettagliata del Codice

- **Cosa succede internamente**: La funzione `print_info` si aspetta un oggetto di tipo `work`. Quando passiamo un `book`, il compilatore crea un nuovo oggetto `work` copiando solo la parte "base" di `book` (ID e Titolo) e ignorando tutto il resto (Autore e Anno).
- **Perdita di Polimorfismo**: Poiché l'oggetto dentro la funzione è ora un vero `work` e non più un `book`, la chiamata a `print()` eseguirà la versione della base, anche se avevamo passato un libro.
- **Conclusione**: Il polimorfismo dinamico non funziona con il passaggio per valore.

7.2 La Soluzione: Riferimenti e Puntatori

Per preservare l'identità dell'oggetto originale e permettere al polimorfismo di agire, dobbiamo usare riferimenti (`&`) o puntatori (`*`).

Listing 12: Passaggio polimorfico corretto

```
// Funzione che accetta un riferimento costante alla classe base
void print_correct(work const& w) {
w.print(); // Polimorfismo attivo!
}

int main() {
book b{"001", "Dune", "Frank Herbert", 1965};
print_correct(b); // Funziona correttamente
}
```

7.2.1 Spiegazione Dettagliata del Codice

- **Nessuna Copia**: `work const& w` non crea un nuovo oggetto, ma è solo un "alias" (un soprannome) per l'oggetto originale.
- **Late Binding**: Poiché stiamo agendo sull'oggetto originale attraverso un riferimento alla base, il C++ consulterà la *vtable* ed eseguirà `book::print()`.

7.3 Interfacce e Classi Astratte (Approfondimento)

Il materiale sottolinea come le classi con funzioni virtuali pure fungano da Contratti.

- **Contratto**: La classe base definisce *cosa* deve essere fatto (es. "ogni opera deve potersi stampare").
- **Implementazione**: Le classi derivate definiscono *come* farlo.

7.4 Esercitazione: Progettazione di `report`

Per verificare la bontà del design, il manuale propone di aggiungere una nuova classe `report` (un'opera con titolo e istituzione rilasciante).

Listing 13: Aggiunta della classe report

```
class report : public work {
public:
report(std::string_view id, std::string_view title,
std::string_view institution)
: work{id, title}, institution_{institution} {}

void print() const override {
    std::cout << "Report: " << title() << "[Istituto:" << institution_
    << "]" << std::endl;
}
private:
std::string institution_;
};
```

7.4.1 Osservazione Critica sulla Manutenibilità

Se il codice è stato scritto bene (usando il polimorfismo e contenitori di puntatori alla base), l'aggiunta di `report` richiederà solo la definizione della classe stessa. Il resto del programma (funzioni di stampa, gestione del catalogo) non dovrà subire modifiche, dimostrando la potenza del design orientato agli oggetti.

8 Best Practices e Progettazione Avanzata

Nella fase finale dello studio della programmazione orientata agli oggetti, è fondamentale comprendere non solo come scrivere il codice, ma come scriverlo in modo che sia robusto e facilmente estensibile.

8.1 Regole d'oro per i Distruttori

Come abbiamo accennato in precedenza, la gestione della memoria è il punto più critico quando si usa il polimorfismo. Il materiale sottolinea una regola fondamentale che ogni programmatore C++ deve seguire.

Listing 14: Regola del Distruttore Virtuale

```
class base {
public:
// Dichiarare sempre il distruttore come virtual nelle classi base
virtual ~base() = default;
};
```

8.1.1 Spiegazione Dettagliata del Codice

- **Innocuità del Distruttore**: Se una classe non gestisce risorse dinamiche (come puntatori grezzi o file aperti), il distruttore generato dal compilatore è sufficiente. Tuttavia, deve essere comunque `virtual`.
- **Mancanza di Vincoli del Compilatore**: È importante notare che il compilatore C++ non obbliga a dichiarare un distruttore virtuale. Si tratta di una disciplina del programmatore per evitare errori subdoli di corruzione della memoria che si manifesterebbero solo a runtime.

8.2 Esercitazione Pratica: Pretty Print di un file CSV

Il manuale propone un esercizio conclusivo che integra tutti i concetti visti: ereditarietà, polimorfismo, classi astratte e gestione dei file.

Obiettivo: Leggere un file di testo in formato CSV (valori separati da punto e virgola) contenente dati di libri, periodici e film, e stamparli in modo formattato.

Listing 15: Esempio di parsing e gestione polimorfica

```
// Esempio di logica di lettura file
void process_file(std::string_view filename) {
std::ifstream file(filename.data());
std::string line;
std::vector<std::unique_ptr<work>> catalog;

while (std::getline(file, line)) {
    // Logica di parsing (semplificata)
    if (line.starts_with("book")) {
        // Estrai i campi e aggiungi al catalogo
        catalog.push_back(std::make_unique<book>(...));
    } else if (line.starts_with("movie")) {
        catalog.push_back(std::make_unique<movie>(...));
    }
}

// Stampa finale polimorfica
for (const auto& item : catalog) {
    item->print();
}
}
```

8.2.1 Spiegazione Dettagliata del Codice

- **Lettura Linea per Linea**: Viene suggerito l'uso di `std::getline` per leggere il file una riga alla volta, memorizzandola in una `std::string`.
- **Parsing Dinamico**: Si utilizzano i metodi della classe `std::string` per analizzare il contenuto della riga. Il primo campo della riga determina quale tipo di oggetto creare (`book`, `periodical` o `movie`).
- **Pooling degli Oggetti**: Tutti gli oggetti creati vengono inseriti in un unico contenitore (`std::vector`), sfruttando il polimorfismo per trattarli uniformemente.

8.3 Verifica del Design: Estensibilità

L'ultima pagina del materiale pone una domanda cruciale per valutare la qualità del lavoro svolto: *"Immagina che sia necessaria una nuova classe **report**. Quante parti del tuo codice dovresti cambiare?"*.

- **Design Ottimale**: Se il codice è progettato correttamente secondo i principi OOP, dovresti aggiungere solo la definizione della classe `report` e una singola riga nella logica di parsing del file.
- **Design Rigido**: Se invece devi modificare il ciclo di stampa o altre funzioni del programma, significa che il polimorfismo non è stato sfruttato appieno.

Osservazione finale: Il vero valore della programmazione orientata agli oggetti non risiede nella complessità, ma nella capacità di rendere il software "resiliente al cambiamento". Un sistema dove l'aggiunta di una funzionalità richiede la modifica di una sola riga è un sistema professionale e manutenibile.

8.4 Curiosità: cosa è `std::ostream&`

Per capire cosa rappresenti `std::ostream&`, dobbiamo analizzare i tre componenti che lo costituiscono: il namespace, la classe e il modificatore di riferimento.

8.4.1 Scomposizione Tecnica

1. `std::` È il namespace (spazio dei nomi) della libreria standard del C++. Indica che la classe `ostream` definita successivamente appartiene alla collezione di strumenti standard del linguaggio, evitando conflitti con eventuali altre classi chiamate `ostream` scritte da altri programmatori.
2. `ostream`: È il nome della classe, abbreviazione di *"output stream"* (flusso di output). È una classe definita nell'header `<ostream>` (incluso solitamente tramite `<iostream>`) che gestisce la scrittura sequenziale di dati verso una destinazione.
3. `&`: Indica un riferimento (reference). In C++, passare un oggetto tramite `&` significa che non stiamo creando una copia del flusso, ma stiamo lavorando direttamente sull'oggetto originale.

8.4.2 Il Concetto di "Stream" (Flusso)

Uno "stream" è un'astrazione logica di un canale di comunicazione. Immagina un tubo dove i dati entrano da un lato (il tuo programma) ed escono dall'altro (il monitor, un file, o una stampante).

- **Destinazioni diverse, stessa interfaccia:** La potenza di `std::ostream` è che non gli importa dove finiscano i dati. `std::cout` è un'istanza di `std::ostream` che punta al video, ma potresti avere un `std::ofstream` che punta a un file: entrambi si comportano nello stesso modo.

8.4.3 Perché il Riferimento (&) è Obbligatorio?

In C++, gli oggetti di tipo stream (come `ostream`) hanno una caratteristica fondamentale: non sono copiabili.

- **Motivazione Tecnica:** Non avrebbe senso "copiare" un canale di comunicazione. Se hai un file aperto, non puoi avere due oggetti diversi che credono di possedere e gestire lo stesso file contemporaneamente in modo indipendente.
- **Conseguenza Pratica:** Poiché non si possono copiare, devono essere passati alle funzioni esclusivamente tramite riferimento (`&`).

8.4.4 Esempio di Utilizzo: Overloading dell'operatore «

L'uso più comune di `std::ostream&` in un contesto universitario è la creazione di funzioni che permettono di stampare oggetti personalizzati.

Listing 16: Overloading dell'operatore di inserimento

```
#include <iostream>
#include <string>

class Student {
public:
std::string name = "Mario";
int id = 12345;
};

// Questa funzione accetta un riferimento a un ostream e restituisce
lo stesso riferimento
std::ostream& operator<<(std::ostream& os, const Student& s) {
// Usiamo 'os' invece di 'cout' per rendere la funzione universale
os << "Studente: " << s.name << " (Matricola: " << s.id << ")";

// Restituiamo il flusso per permettere il concatenamento
(es. << s << endl;)
return os;
}
```

8.4.5 Spiegazione Dettagliata del Codice

- **Parametro** `std::ostream& os`: Questo permette alla funzione di funzionare con **qualsiasi** flusso di output. Se la chiami con `std::cout « s;`, `os` diventerà un riferimento a `cout`. Se la chiami con un file, `os` scriverà sul file.

- **Valore di Ritorno** `std::ostream&`: La funzione restituisce il flusso stesso. Questo è ciò che permette di scrivere `cout « oggetto1 « oggetto2 « endl;`. Ogni operazione restituisce il flusso al passaggio successivo (il cosiddetto *chaining*).

Osservazione conclusiva: In sintesi, `std::ostream&` è il modo universale e sicuro in C++ per dire al compilatore: "Voglio accedere a un canale di output esistente per scriverci dentro dei dati, senza creare inutili e impossibili copie del canale stesso".